编译总结感想

> 23373334 彭子峻

延续了一个学期的编译开发，终于告一段落了。整个开发过程，可谓是有喜有悲吧。

编译作业，是我为数不多个人独立开发较大型项目的经验（10k+ lines）。整体架构借鉴了多方设计，最后综合出了一套于我而言最优的架构设计。这对培养我独自开发项目的自信，客观而言是有提升的。

期间我最满意的部分，是我基本复现了*Building an Optimizing Compiler* 一文中的寄存器分配思路。其不需要区分全局/局部可用的寄存器，改进了图着色算法，在局部变量的分配中亦引入了图着色算法，且允许局部变量复用不与之冲突的全局寄存器，大大提升了寄存器的利用率。对于完成之的过程，我最大的感想是两个：

- **尽信书不如无书**。一开始，我尝试完全复现原文中提供的伪代码，但在复现过程中，发现原文的伪代码存在诸多错误。最后还是需要自己吃透原文的思路，重新按其思路写一遍算法。
- **没有银弹**。从时间而言，完全开发完这套分配逻辑，花费了我一周有余的时间；从最终效果而言，这一算法虽能尽可能减少溢出，但其不区分全局/局部变量可用寄存器，允许局部变量复用全局变量用到寄存器的思路，**带来了函数调用前后，必须保存所有用到寄存器的访存开销**。

开发过程，亦让我意识到，**计算机体系结构有关知识，对于编译等计算机体系结构强相关领域**的重要性。在开发期间，我就已注意到，MARS模拟器允许我们使用32个浮点寄存器，而我们编译开发的任何过程均不涉及浮点运算。这为我的优化实现带来了一个相当有用的思路。

开发过程有一大遗憾，**我原计划让我的编译器，原生即支持多维数组。** 我已在前端部分提供了这一部分的支持，但是在后续完成后端和优化时，我认为这么做复杂度实在太高，遂放弃之。**过度设计，看来有时并不是好事。**

此外，优化过程的实现，让我意识到，**一开始选对方向，远比一味努力修补有问题的思路重要。**这是在我实现函数内联优化时发现的。我初始的实现，仅是递归将函数代码嵌入到源代码中，在出现调用环路时终止展开，期间未有做任何常量折叠。对于testcase7中出现的递归调用，我希望通过重复上述过程+常量折叠/控制流简化的方法，实现对其完全展开，但最后发现其效率奇低，完全展开testcase7的时候，一定会导致testcase2运行超时。**其实还有一个思路，是局部解释执行，不过此时我已积重难返，实在无心实现这种办法了**。算是一大遗憾吧。

最后，我还意识到，我在编译课上所学，仅仅是编译这一课题的冰山一角。现代体系结构的向量化等优化，均需要编译器后端提供有力的支持，而这些难度自然过大，不可能放在时间有限的编译课程内。希望我在日后，能有机会自行探索这些有趣的事情吧。